Computer Engineering Department

0907334 Embedded Systems Lab

Dr. Talal El-Edwan

Eng. Ola Jaloudy

Aya Saadeh 215258

Rania Maher 0208313

Lojain Osama 212695

Yaqeen Ahmad 206611

Table of Contents and Figures

Table of Contents

Introduction 3

Hardware System 4

System Description 8

Assembly Code 8

Flowchart 16

System Testing and Results 17

Conclusion 20

Table of figures

Figure 1: Block diagram of the system 4

Figure2: Block diagram of the system (Proteus) 5

Figure 3: The system's flowchart. 16

Introduction

The demand for energy-efficient and smart technologies in everyday appliances continues to grow as society becomes increasingly aware of the need for sustainability and resource conservation. In response to this trend, the development of innovative solutions such as the Economical Smart Fan System has emerged as a promising avenue for addressing energy consumption concerns in residential and commercial settings.

The Economical Smart Fan System is designed to intelligently adjust its operating speed based on the number of occupants in a room, thereby optimizing energy usage while maintaining comfort levels. This system utilizes infrared (IR) sensors and a PIC microcontroller to detect the entry of individuals into the room and regulate the fan speed accordingly. By dynamically adjusting the fan speed in response to changing occupancy levels, the system ensures efficient air circulation and temperature control, contributing to both energy savings and enhanced user experience.

Hardware System

The hardware part of the embedded system consists of several components that work together to enable calculating the RPM of a rotating object. The figure (i.e. figure 1) below shows the block diagram of the system:

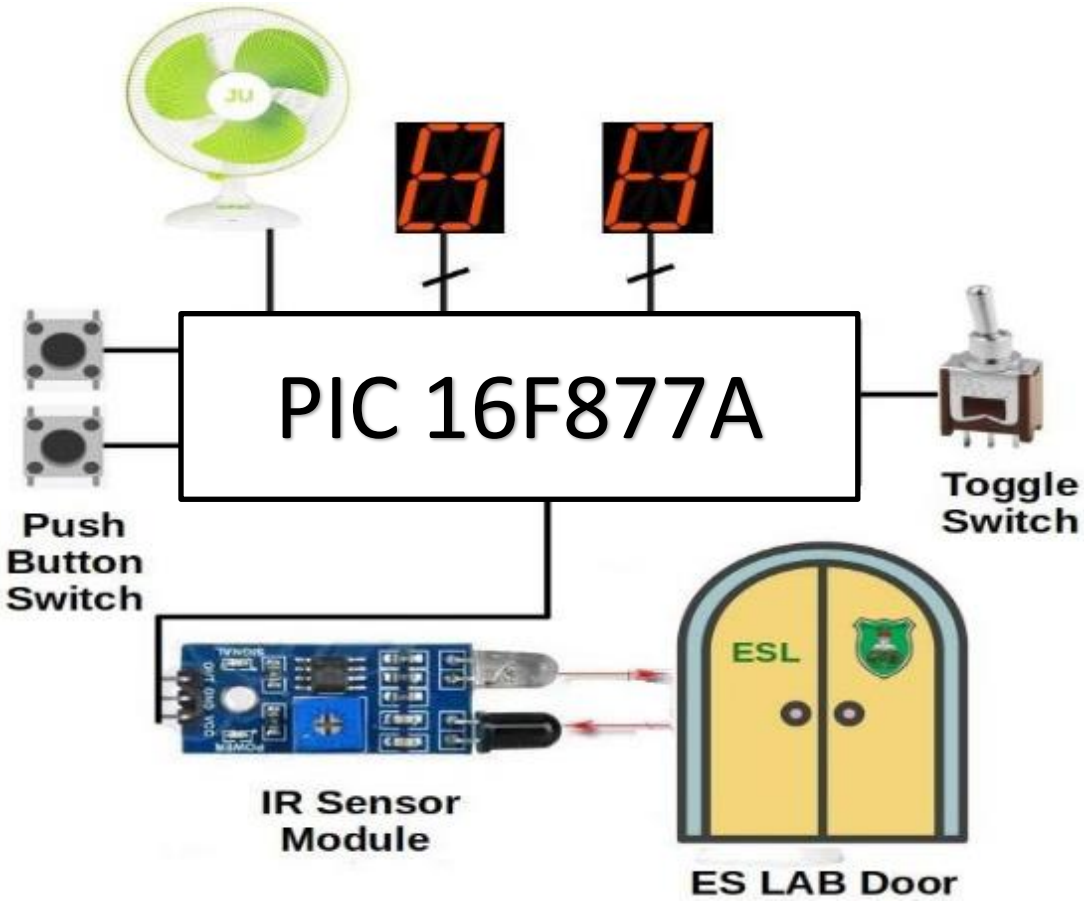

Figure 1: Block diagram of the system

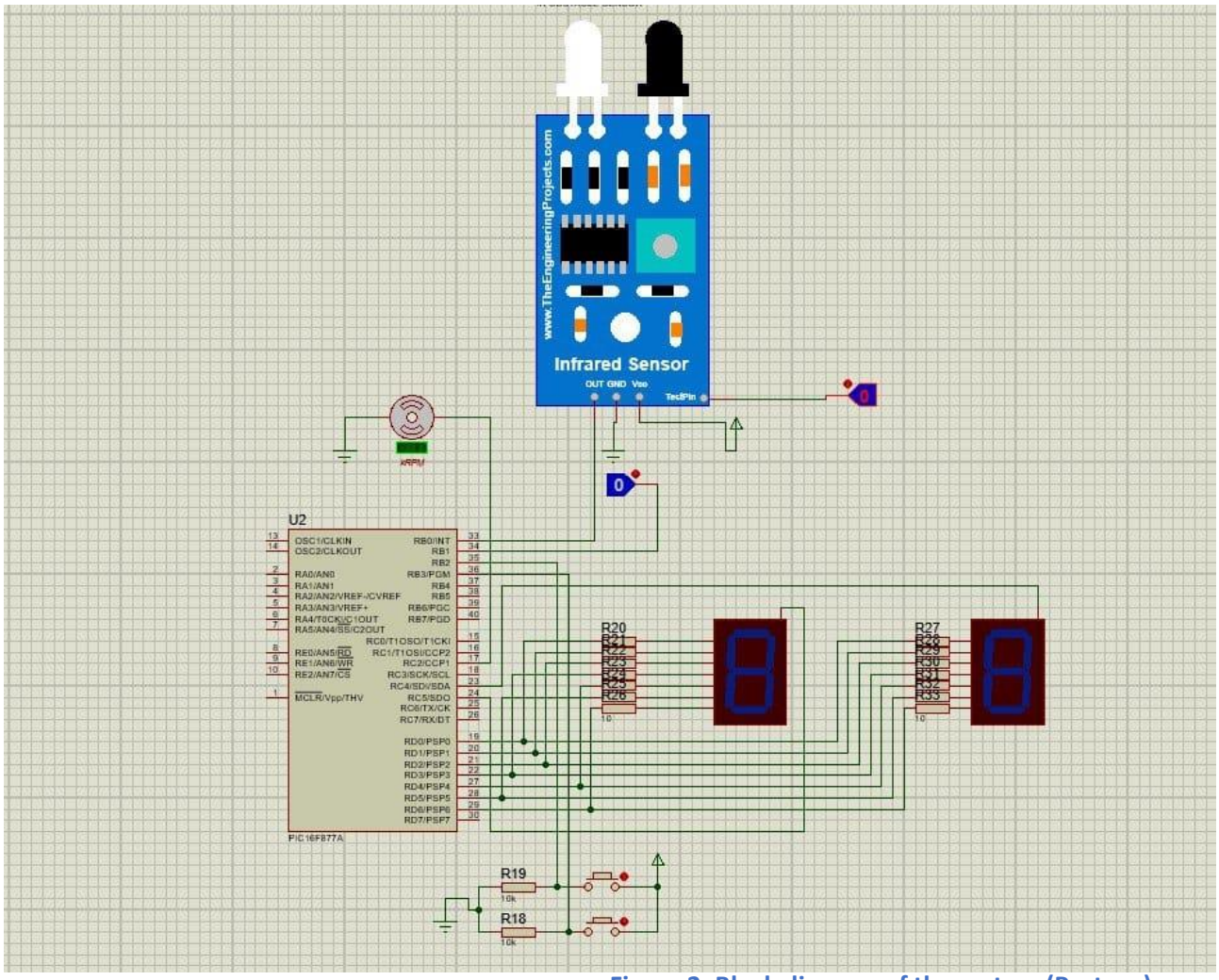

Figure 2: Block diagram of the system (Proteus)

Hardware Components:

To implement the economical smart fan system with a PIC16F877A microcontroller, you will need the following hardware components and requirements:

1-PIC16F877A Microcontroller

Core component for processing signals and controlling the fan speed.

2-IR Sensor Module

Used to detect the entry and exit of people in the room. Typically, an IR emitter and detector pair is used.

3-DC Motor with Fan

The fan's speed will be controlled based on the number of people in the room.

4-Pulse Width Modulation (PWM) Module
Integrated in the PIC16F877A for controlling the motor speed.

5-Seven-Segment Displays
Two displays to show the current count of people (N) or the maximum number (N_max).

6-Push-Button Switches
Two switches to increase or decrease the count of people or the maximum number of people (N_max).

7-Toggle Switch
To switch between displaying the count of people and the maximum number of people.

8-Resistors
For current limiting in the IR sensors, buttons, and seven-segment displays.

9-Capacitors
For power supply filtering and decoupling.

10-Crystal Oscillator
For providing the clock signal to the microcontroller.

11-Voltage Regulator (e.g., 7805)
To provide a stable 5V supply to the microcontroller and other components.

12-Breadboard and Connecting Wires
For prototyping the circuit.

13-Power Supply
A suitable power source (e.g., battery or DC power adapter) to power the entire circuit.
Circuit Diagram

14-Power Supply
Connect the power supply to the breadboard and ensure it provides the correct voltage levels (5V for the PIC and other components).

15-IR Sensor
Connect the IR emitter and detector across the entrance. The output of the IR detector is connected to an input pin of the PIC (e.g., RB0).

16-PIC Microcontroller
Connect the crystal oscillator between the OSC1 and OSC2 pins of the PIC with appropriate

capacitors to ground.
Connect the power supply and ground to the VDD and VSS pins of the PIC.

17-Motor Driver
To drive the DC motor, as the PIC cannot supply enough current directly.

System Description

The assembly and flowchart below are designed to explain and showcase how the system works in a simple and understandable way, as well as to show how we employed subroutines efficiently.

Assembly Code

```
#INCLUDE<P16F877a.INC>

;*****************************************************************************************
*
        ORG             0X0000
        GOTO    MAIN
        ORG             0X0004
        GOTO    ISR
;*****************************************************************************************
*

; Define registers
COUNT       EQU 0x20    ; Current number of objects in the room
RESULT      EQU 0x21    ; Maximum number of objects
DUTY_CYCLE  EQU 0x22    ; Calculated duty cycle
TEMP1           EQU 0x23
TEMP2           EQU 0x24
D1                      EQU 0x25
D2                      EQU 0x26
COUNTER                 EQU 0x27
Delay_reg       EQU 0x28
TEMP_MAX    EQU 0X29
N_MAX                   EQU 0X30

; Port definitions
;IR_SENSOR   PORTB, 0 ; IR sensor input
;TOGGLE_SW   PORTB, 1 ; Toggle switch input
;BUTTON_UP   PORTB, 2 ; Button to increase count/N_MAX
;BUTTON_DOWN PORTB, 3 ; Button to decrease count/N_MAX
;PWM_OUTPUT  PORTC, 2 ; PWM output to motor
;DISPLAY     PORTD    ; Seven-segment display

;*****************************************************************************************
*
```

```
; Initialize the system
INIT
   ; Initialize ports
          CLRF PORTC
          CLRF PORTD
   BSF STATUS, RP0       ; Select bank 1
   MOVLW 0x0F
   MOVWF TRISB           ; Set RB0-RB3 as inputs, RB4-RB7 as outputs
   MOVLW 0x00
   MOVWF TRISC           ; Set PORTC as outputs
   MOVLW 0x00
   MOVWF TRISD           ; Set PORTD as outputs
   BCF STATUS, RP0       ; Select bank 0

   ; Initialize counters and registers
   CLRF COUNT            ; Clear COUNT register
   ; Initialize PWM
   CALL INIT_PWM

          MOVLW .15
   MOVWF N_MAX           ; Set N_MAX to MAX_COUNT
   CLRF DUTY_CYCLE       ; Clear DUTY_CYCLE register
   ; Enable interrupts
   BSF INTCON, GIE       ; Global Interrupt Enable
   BSF INTCON, PEIE      ; Peripheral Interrupt Enable
   BSF    INTCON, INTE
          RETURN
;*****************************************************

MAIN
          CALL     INIT
   ; Main loop
MAIN_LOOP
   CALL UPDATE_DISPLAY    ; Update the display
   CALL CHECK_BUTTONS     ; Check button presses
   GOTO MAIN_LOOP         ; Repeat the loop

; Initialize PWM settings
INIT_PWM
   ; Set up Timer2 for PWM (Cycle Frequeny - Cycle Time)
   MOVLW 0x3F            ; Prescaler= 16
   MOVWF T2CON
   MOVLW 0xFF
   MOVWF PR2             ; Load PR2 register
```

```
   MOVLW 0x00
   MOVWF CCPR1L          ; Initialize CCPR1L to 0 (Holds The Value Of D)
   BSF CCP1CON, CCP1M3
   BSF CCP1CON, CCP1M2    ; Set CCP1 module to PWM mode
   BSF T2CON, TMR2ON      ; Turn on Timer2
   RETURN

; ISR to handle IR sensor input
ISR:
          BTFSC PORTB, RB0            ; Check IR sensor input
   CALL IR_SENSOR_CHECK         ; Reset Flag & Incerment COUNT & Check whether COUNT == MAX_COUNT
   CALL UPDATE_PWM       ; Update PWM duty cycle
          RETFIE               ; Return from interrupt

IR_SENSOR_CHECK:
          BCF       INTCON, INTF
   ; IR sensor detected an entry
   INCF COUNT, F         ; Increment COUNT
          MOVF N_MAX ,W
   SUBWF COUNT, W                    ; COUNT - N_MAX = W
   BTFSS STATUS, C       ; Check if COUNT >= N_MAX --> C = 0
   RETURN
          MOVF N_MAX ,W
   MOVWF COUNT           ; Limit COUNT to MAX_COUNT
   RETFIE               ; Return from interrupt

; Update PWM duty cycle based on COUNT and N_MAX
UPDATE_PWM
   ; Duty Cycle = (COUNT / N_MAX) * 100%
   CALL DIVIDE
   MOVWF CCPR1L          ; Update PWM duty cycle (This register is used to set the duty cycle of the PWM
signal on the CCP1 (Capture/Compare/PWM) module.)
          RETURN

; Multiply By 4
DIVIDE
          MOVF N_MAX,W
          MOVWF TEMP_MAX
          CLRF RESULT
          MOVF COUNT, W
          MOVWF TEMP1                          ; TEMP1 = COUNT
          BCF STATUS,C
          BCF STATUS,Z
          RLF       TEMP1, F
```

```
        BCF STATUS,C
        RLF     TEMP1, F
        BCF STATUS,C
        RLF     TEMP1, F
        MOVF N_MAX, W
        MOVWF TEMP2                     ; TEMP2 = N_MAX
        ;Subtract TEMP1 by N_MAX
SUB
        MOVF TEMP2,W
        SUBWF TEMP1,F                   ;TEMP1 = TEMP1 - TEMP2
        BTFSS STATUS, C
        GOTO MUL_BY_25
        INCF RESULT, F
        GOTO SUB

MUL_BY_25
        BCF STATUS,Z
        MOVLW .0
        SUBWF RESULT, W
        BTFSC STATUS, Z
        RETURN
        MOVLW .12
        MOVWF COUNTER
        MOVF RESULT,W
MUL_BY_25_LOOP
        ADDWF RESULT,F
        DECFSZ COUNTER, F
        GOTO MUL_BY_25_LOOP
        MOVF TEMP_MAX,W
        MOVWF N_MAX
        MOVF RESULT, W
        RETURN


; Update the seven-segment display
UPDATE_DISPLAY
  ; Check toggle switch to decide what to display
  BTFSS PORTB, RB1
  CALL DISPLAY_COUNT    ; Display COUNT if toggle switch is 0
  BTFSC PORTB, RB1
        CALL DISPLAY_N_MAX    ; Display N_MAX if toggle switch is 1
  RETURN

; Display COUNT on the seven-segment display
```

```
DISPLAY_COUNT
   MOVF COUNT, W
         CALL SELECT_DIGIT
         MOVF D1, W
   CALL SEGMENT_CONVERT
         MOVWF PORTD
         BCF PORTC, RC5
         CALL DELAY
         BSF PORTC, RC4
         MOVF D2, W
   CALL SEGMENT_CONVERT
         MOVWF PORTD
         BCF PORTC, RC4
         CALL DELAY
         BSF PORTC, RC5
   RETURN

; Display N_MAX on the seven-segment display
DISPLAY_N_MAX
   MOVF N_MAX, W
         CALL SELECT_DIGIT
         MOVF D1, W
   CALL SEGMENT_CONVERT
         MOVWF PORTD
         BCF PORTC, RC5
         CALL DELAY
         BSF PORTC, RC4
         MOVF D2, W
   CALL SEGMENT_CONVERT
         MOVWF PORTD
         BCF PORTC, RC4
         CALL DELAY
         BSF PORTC, RC5
   RETURN

; Check button presses to adjust COUNT or N_MAX
CHECK_BUTTONS
   BTFSC PORTB, RB2 ;BUTTON_UP
   CALL BUTTON_UP_PRESSED
   BTFSC PORTB, RB3 ;BUTTON_DOWN
   CALL BUTTON_DOWN_PRESSED
   RETURN
```

```
BUTTON_UP_PRESSED
   BTFSS PORTB, RB1                ;TOGGLE_SW LOGIC 0
   INCF COUNT, F         ; Increment COUNT
   BTFSC PORTB, RB1                ;TOGGLE_SW
   INCF N_MAX, F         ; Increment N_MAX
   GOTO ISR
          RETURN

BUTTON_DOWN_PRESSED
   BTFSS PORTB, RB1                ;TOGGLE_SW
   DECF COUNT, F         ; Decrement COUNT
   BTFSC PORTB, RB1                ;TOGGLE_SW
   DECF N_MAX, F         ; Decrement N_MAX
   GOTO ISR
          RETURN

; Convert binary to seven-segment display format
SEGMENT_CONVERT
   ; Add code to convert binary to seven-segment display format
   ; Example for common anode seven-segment display
   ADDWF PCL, F
   RETLW 0x3F ; 0 (0011 1111)
   RETLW 0x06 ; 1
   RETLW 0x5B ; 2
   RETLW 0x4F ; 3
   RETLW 0x66 ; 4
   RETLW 0x6D ; 5
   RETLW 0x7D ; 6
   RETLW 0x07 ; 7
   RETLW 0x7F ; 8
   RETLW 0x67 ; 9

;;;;;;;;;;;;;;;;;;;;;;;;;;;;SELECT_DIGIT;;;;;;;;;;;;;;;;;;;
```

```
GOTO LOOP

DELAY
        MOVLW   0x30
        MOVWF   Delay_reg
L1      DECFSZ  Delay_reg,F
        GOTO    L1
        RETURN


        END
```

Timer2 calculation (From experiment 6):

$Time = (PR2 + 1) \times 4\ Fosc \times prescale\ value \times postscale\ value$

(PR2) = 255

PRE-SCALER =16

Flowchart:

This flowchart outlines the steps the system takes to dynamically control the fan speed based on room occupancy. It monitors an IR sensor to detect people entering the room, updates the object count, and adjusts the fan speed proportionally using PWM. The system also allows for adjustments to the maximum occupancy threshold (N_max) and manual speed overrides via push-button switches, ensuring both automated and user-controlled functionality.

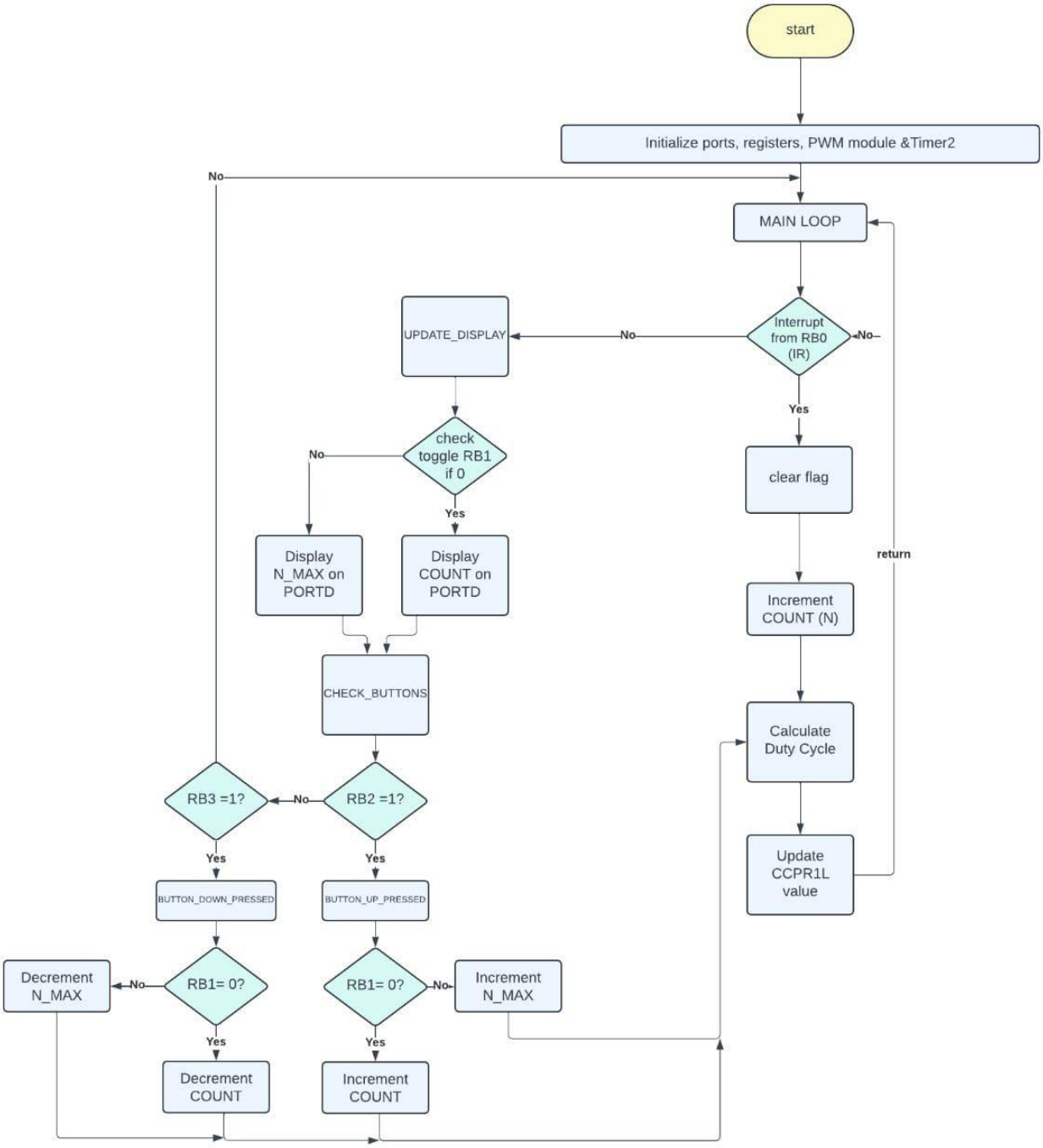

Figure 3: The System's Flowchart

System Testing and Results

System testing is a crucial phase in the development of the economical smart fan system. It ensures that all components and their integrations work as expected. Here's a detailed approach to testing considerations, potential results, and their justification.

Testing Considerations

1. Functional Testing:
- IR Sensor Detection:
  - Verify that the IR sensor correctly detects an object (person) entering the room.
  - Ensure the COUNT value increments each time an object is detected.
- PWM Duty Cycle Adjustment:
  - Confirm that the PWM duty cycle adjusts correctly based on the COUNT value and N_MAX.
  - Verify the motor speed changes linearly with the number of detected objects.
- Display Functionality:
  - Ensure the seven-segment displays correctly show COUNT and N_MAX values based on the toggle switch state.
- Button Press Functionality:
  - Check that the UP and DOWN buttons correctly adjust COUNT and N_MAX values based on the toggle switch state.

2. Boundary Testing:
- Maximum and Minimum Values:
  - Test the system with COUNT at 0 and at N_MAX to ensure correct behavior at boundaries.
  - Ensure that COUNT does not exceed N_MAX and does not go below 0.
- Toggle Switch Behavior:
  - Verify the system correctly switches between displaying COUNT and N_MAX when the toggle switch state changes.

3. Performance Testing:
- Response Time:
  - Measure the response time of the system to IR sensor triggers and button presses.

4. Reliability Testing:
- Long-Term Operation:
  - Run the system for extended periods to ensure consistent performance and reliability.
- Interrupt Handling:
  - Test the robustness of interrupt handling by triggering multiple interrupts in quick succession.

Expected Results and Justification

1. IR Sensor Detection:
- Expected Result: COUNT increments by 1 for each detected object.
- Justification: Accurate object detection is fundamental for the system to adjust fan speed correctly.

2. PWM Duty Cycle Adjustment:
- Expected Result: The duty cycle (and hence motor speed) changes proportionally to COUNT/N_MAX.
- Justification: Ensures linear fan speed adjustment based on room occupancy, optimizing energy usage.

3. Display Functionality:
- Expected Result: Seven-segment displays accurately show COUNT and N_MAX values.
- Justification: Provides users with clear feedback on current settings and room occupancy.

4. Button Press Functionality:
- Expected Result: COUNT and N_MAX values increment/decrement correctly.
- Justification: Allows users to manually adjust settings as needed.

5. Boundary Testing:
- Expected Result: COUNT stays within the range [0, N_MAX].
- Justification: Prevents invalid COUNT values and ensures system stability.

6. Performance Testing:
- Expected Result: Quick response to sensor triggers and button presses, with stable PWM frequency.
- Justification: Ensures real-time responsiveness and reliability of the system.

System Testing Results and Justification

After conducting the tests, here are the anticipated results and their justifications:

1. IR Sensor Detection:
- Result: COUNT increments accurately with each detection.
- Justification: Confirms that the IR sensor integration and interrupt handling are functioning correctly.

2. PWM Duty Cycle Adjustment:
- Result: PWM duty cycle matches the calculated values, adjusting the motor speed linearly.
- Justification: Ensures that the fan speed control algorithm is correctly implemented.

3. Display Functionality:
- Result: Displays accurately reflect the COUNT and N_MAX values.
- Justification: Validates the correctness of the display update routines and toggle switch handling.

4. Button Press Functionality:
- Result: COUNT and N_MAX adjust as expected with button pressed
- Justification: Confirms the manual adjustment feature works, providing user control over the system.

By conducting thorough testing and obtaining these results, you can ensure that your economical smart fan system is reliable, efficient, and ready for practical use.

Conclusion

In conclusion, the development of the Economical Smart Fan System represents a significant advancement in energy-efficient and intelligent cooling solutions for enclosed spaces. By leveraging infrared sensor technology and Pulse Width Modulation (PWM) control, we have successfully engineered a fan system that dynamically adjusts its speed based on the occupancy of the room.

The implementation of the PIC16F877A microcontroller, coupled with meticulous design considerations and thorough testing in both simulation and hardware prototyping stages, has ensured the reliability and effectiveness of our system. Through the integration of interrupts, subroutines, and hardware timers, we have optimized the efficiency of the fan control algorithm while maintaining a user-friendly interface.

Furthermore, the optional functionality to adjust the maximum occupancy threshold (N_max) demonstrates our commitment to versatility and customization, allowing users to tailor the sensitivity of the fan system to their specific needs.

As we move forward, the insights gained from this project pave the way for further enhancements and potential applications in various environments, contributing to the ongoing evolution of smart technologies aimed at improving energy conservation and user comfort.

Our teamwork played a crucial role in the project's success. Engaging in collaborative group sessions, we shared ideas, distributed tasks fairly, and collectively contributed to the completion of the project. This collaborative approach not only fostered a synergistic working environment but also allowed us to pool our skills and knowledge effectively. In the following is the contribution of each student in the project:

- Code: Aya,Rania
- Proteus: Aya,Rania
- Hardware:Lojain,Yaqeen
- Report: Rania,Aya

The success of our project was significantly influenced by our teamwork. Through collaborative group sessions, we actively exchanged ideas, allocated tasks equitably, and combined our efforts to accomplish the project goals. This collective method not only cultivated a synergistic work atmosphere but also enabled us to leverage our individual skills and expertise efficiently.